LAB 13.3 Transformer – Hugging Face – BERT

24O3A52O31

Akula Manoj

Install Required Libraries

```
!pip install transformers datasets -q

import torch
import numpy as np
import pandas as pd

from datasets import load_dataset
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    TrainingArguments,
    Trainer
)

from sklearn.metrics import accuracy_score, precision_recall_fscore_support, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt
```

Load Dataset (IMDb)

```
"""
STEP 3: Load IMDb Dataset
Binary Sentiment Classification (Positive / Negative)
"""


dataset = load_dataset("imdb")

print(dataset)
print("\nSample Record:\n")
print(dataset["train"][0])
```

```
DatasetDict({
    train: Dataset({
        features: ['text', 'label'],
        num_rows: 25000
    })
    test: Dataset({
        features: ['text', 'label'],
        num_rows: 25000
    })
    unsupervised: Dataset({
        features: ['text', 'label'],
        num_rows: 50000
    })
})

Sample Record:

{'text': 'I rented I AM CURIOUS-YELLOW from my video store because of all the controversy that surrounded it when it was fir
```

Check Class Distribution

```
"""
STEP 4: Check Class Distribution
"""

train_labels = dataset["train"]["label"]

unique, counts = np.unique(train_labels, return_counts=True)

print("Class Distribution:")
for u, c in zip(unique, counts):
    print(f"Label {u}:", c)
```

```
Class Distribution:
Label 0: 12500
Label 1: 12500
```

Load BERT Tokenizer

```
"""
STEP 5: Load Pretrained BERT Tokenizer
"""

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
```

```
config.json: 100%  570/570 [00:00<00:00, 58.7kB/s]
tokenizer_config.json: 100%  48.0/48.0 [00:00<00:00, 4.82kB/s]
vocab.txt: 100%  232k/232k [00:00<00:00, 11.3MB/s]
tokenizer.json: 100%  466k/466k [00:00<00:00, 974kB/s]
```

Tokenization Function

```
"""
STEP 6: Tokenize Text
Apply padding and truncation
"""

def tokenize_function(example):
    return tokenizer(
        example["text"],
        padding="max_length",
        truncation=True,
        max_length=128
    )

tokenized_dataset = dataset.map(tokenize_function, batched=True)
```

```
Map: 100%  25000/25000 [00:43<00:00, 461.10 examples/s]
Map: 100%  25000/25000 [00:31<00:00, 1185.87 examples/s]
Map: 100%  50000/50000 [00:45<00:00, 1095.70 examples/s]
```

Prepare Dataset for PyTorch

```
"""
STEP 7: Convert Dataset to PyTorch Format
"""

tokenized_dataset = tokenized_dataset.remove_columns(["text"])
tokenized_dataset = tokenized_dataset.rename_column("label", "labels")
tokenized_dataset.set_format("torch")

train_dataset = tokenized_dataset["train"]
test_dataset = tokenized_dataset["test"]
```

Load Pretrained BERT Model

```
"""
STEP 8: Load BERT Model for Classification
"""

model = AutoModelForSequenceClassification.from_pretrained(
    "bert-base-uncased",
    num_labels=2
)
```

```
model.safetensors: 100%                                    440M/440M [00:04<00:00, 200MB/s]

Loading weights: 100%                                      199/199 [00:00<00:00, 559.21it/s, Materializing param=bert.pooler.dense.weight]
BertForSequenceClassification LOAD REPORT from: bert-base-uncased
Key                                         | Status     |
-------------------------------------------+------------+-
cls.predictions.transform.dense.bias        | UNEXPECTED |
cls.predictions.transform.dense.weight      | UNEXPECTED |
cls.seq_relationship.bias                   | UNEXPECTED |
cls.predictions.bias                        | UNEXPECTED |
cls.predictions.transform.LayerNorm.weight  | UNEXPECTED |
cls.seq_relationship.weight                 | UNEXPECTED |
cls.predictions.transform.LayerNorm.bias    | UNEXPECTED |
classifier.weight                           | MISSING    |
classifier.bias                             | MISSING    |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING   :those params were newly initialized because missing from the checkpoint. Consider training on your downstream t
```

Define Evaluation Metrics

```
"""
STEP 9: Define Evaluation Metrics
Accuracy, Precision, Recall, F1
"""

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    predictions = np.argmax(logits, axis=1)

    precision, recall, f1, _ = precision_recall_fscore_support(
        labels, predictions, average="binary"
    )

    acc = accuracy_score(labels, predictions)

    return {
        "accuracy": acc,
        "precision": precision,
        "recall": recall,
        "f1": f1
    }
```

Define Training Arguments

```
training_args = TrainingArguments(
    output_dir="./results",
    eval_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=2,
    weight_decay=0.01,
    logging_dir="./logs",
    logging_steps=100
)
```

```
`logging_dir` is deprecated and will be removed in v5.2. Please set `TENSORBOARD_LOGGING_DIR` instead.
```

Initialize Trainer

```
"""
STEP 11: Initialize Trainer
"""

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=test_dataset,
    compute_metrics=compute_metrics
)
```

Train Model

```
"""
STEP 12: Train the Model
"""

trainer.train()
```

[3126/3126 30:10, Epoch 2/2]

Epoch	Training Loss	Validation Loss	Accuracy	Precision	Recall	F1
1	0.290527	0.278101	0.888080	0.896259	0.877760	0.886913
2	0.162490	0.347239	0.891760	0.880852	0.906080	0.893288

```
Writing model shards: 100%                                    1/1 [00:08<00:00,   8.01s/it]
Writing model shards: 100%                                    1/1 [00:10<00:00,  10.64s/it]
Writing model shards: 100%                                    1/1 [00:07<00:00,   7.11s/it]
Writing model shards: 100%                                    1/1 [00:06<00:00,   6.12s/it]
Writing model shards: 100%                                    1/1 [00:14<00:00,  14.54s/it]
Writing model shards: 100%                                    1/1 [00:06<00:00,   6.63s/it]
Writing model shards: 100%                                    1/1 [00:52<00:00,  52.71s/it]
TrainOutput(global_step=3126, training_loss=0.24088545602174882, metrics={'train_runtime': 1811.1232,
'train_samples_per_second': 27.607, 'train_steps_per_second': 1.726, 'total_flos': 3406761944801280.0, 'train_loss':
```

Evaluate Model

```
"""
STEP 13: Evaluate Model Performance
"""

results = trainer.evaluate()
print("\nEvaluation Results:\n", results)
```

[1563/1563 03:46]

```
Evaluation Results:
 {'eval_loss': 0.3169800639152527, 'eval_accuracy': 0.89072, 'eval_precision': 0.8872502378686965, 'eval_recall': 0.8952, 'e
```

Generate Confusion Matrix

```
"""
STEP 14: Confusion Matrix Heatmap
"""

predictions = trainer.predict(test_dataset)
pred_labels = np.argmax(predictions.predictions, axis=1)
true_labels = predictions.label_ids

cm = confusion_matrix(true_labels, pred_labels)

plt.figure(figsize=(6,5))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=["Negative", "Positive"],
    yticklabels=["Negative", "Positive"]
)

plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

[1563/1563 15:52]